Twenty-Eight Ways to Measure Nothing: A Failure Catalogue from Instrumenting an Agent Harness

Anonymous Author(s)
Affiliation
Address
email

Abstract

An OS layer for agentic AI will standardise checkpointing, recovery, and observability, and those designs can only be compared if they can be measured. We built the instrument such comparison requires — an observational timeline committed to git, exhaustive replay of held-out tests at every observation, detection attributed by coverage — and we report two things.

First, the instrument's failure record: **twenty-eight defects, of which twenty-three produced a plausible number rather than an error, and all but one were present while a green test suite watched.** They collapse into four mechanisms — ambient-environment dependence, infrastructure conflated with measurement, final-state measurement of event-shaped quantities, and output consumed without asserting its producer — and we show each mechanism operating in public infrastructure, including the official SWE-bench grader. The capstone: after nineteen fixes, every validation gate we had passed, and the headline number was still wrong, because the agent executed on the host while every check validated the measurement path. A gate certifies only the paths it exercises.

Second, what the corrected instrument measures. On the same benchmark instances, under the same harness and recovery policy, one frontier model stack produced **zero regression events across 40 runs** while another produced **140, in 11 distinct breakage incidents across 6 bearing runs** (equal-N Fisher $p = 0.011$) — every recovered event erased by the harness's own rollback and invisible in the final tree. One officially-*resolved* patch carried three of them, two undetected even by the harness that produced it. With the pre-declared recovery-policy contrast unblinded: no recovery leaves more of this damage in the final tree than rollback (paired sign $p = 0.022$) — and rollback resolves fewer tasks, because the verifier that gates it rejects good patches. Regressions during agent runs are a property of the agent regime, not the benchmark, and the final state — all existing evaluation examines — is where the evidence is not.

1 Introduction

The agentic-systems community is converging on an OS layer: shared abstractions for memory, scheduling, checkpointing, recovery. Choosing between abstractions requires evidence, and this paper is about how much harder producing that evidence is than it looks — not in the statistics, but in the plumbing that produces the numbers the statistics consume.

Our motivating question is narrow. When a step in a long-horizon agent run fails verification, deployed harnesses either **repair in place** (hand the model its own broken tree) or **roll back and retry** (reset to the last verified checkpoint). The second discards work and repays cold-cache token

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

prices; its defence is that it leaves less collateral damage. That claim is testable — if you can say, for every run, *when* previously-working behaviour stopped working.

Contributions. (i) The instrument: observational checkpointing on a hidden git ref, exhaustive replay, coverage-based attribution, and a validation regime whose controls include re-measuring archived runs. (ii) A twenty-eight-defect catalogue with the mechanism for each, in a four-class taxonomy whose classes we show operating in public harnesses. (iii) First measurements from the corrected pipeline: across the same 40 instances, two frontier stacks at equal budgets give a 0-versus-140-event contrast in which final-state evaluation sees one event in 184; and the pre-declared recovery-policy contrast, unblinded with its committed analysis, shows rollback keeps final trees clean at a measurable cost in resolution.

2 What has to be measured

A **regression event** is a held-out test that passed at some observation of a run and fails at a later one. Three properties make it hard to measure honestly.

It is an event, not a state. A regression introduced and then repaired leaves no trace at the end of the run — and repair is exactly what a recovery policy does. An instrument that inspects the final artifact, which is how regressions on this benchmark are measured in published work [3], records zero by construction for every recovered event. §5.1 measures how much that misses: in our data, all of it.

Its absence and the instrument's death are the same observation. Zero is what a clean run produces — and what a dead probe, an unmatched parser, and an empty timeline produce. §4 is twenty-eight instances of this.

Detection must be attributed, not co-located. "The harness failed something while the regression was open" credits whichever policy fails most often with the best detection. Attribution needs a causal join through a file the agent changed; we report attributed, co-occurring, and unknown separately, and never let "could not measure" render as any of them.

3 The instrument, and how it is validated

Observational checkpointing. Every mutating tool call commits the tree to a git ref the agent cannot enumerate, from a private index, so the agent's own `git status` and `git diff` are byte-identical with the instrument on or off. Rollbacks and run-end are observed too.

Exhaustive replay. Held-out tests are replayed at *every* observation inside the instance's pinned container. Bisection would assume monotone verdicts; non-monotonicity is the treatment (row C1).

Attribution by coverage, built once per instance at the base commit, identical across arms by construction.

Validation. Five pre-set gates (negative control, positive control including *recovered* injections, flake screen, unknown-rate ceiling, baseline liveness), plus the control the git substrate makes possible: **re-scoring** — an archived run's committed timeline re-measured under a changed instrument, no model calls, only the instrument varying. This control, exercised twice on different archived runs, reproduced their episode counts exactly each time — once localising a disagreement to the runs rather than the detector, once proving that forty fresh runs' zero (§5.1) belonged to the runs.

4 Twenty-eight ways to measure nothing

S = silent: produced a plausible number rather than an error. **G** = present while the test suite was green (A4 *is* the suite, marked —). **H** = findable only on a clean host. The census closes at submission; defects caught after it are described where they were found and not counted here.

4.1 The catalogue, by mechanism

Class A — ambient-environment dependence. The code asks the machine a question and gets a different answer on a different machine.

#	Defect	Consequence	S	G	H
A1	Shadow commits inherit the machine's git identity	timeline silently empty on any clean machine	✓	✓	✓
A2	Gate probe invoked bare `python`	every probe exit 127 on clean Ubuntu	✗	✓	✓
A3	Unpinned SDK resolved a different major version	two machines ran different code	✗	✓	✓
A4	Test fixtures verified with bare `pytest` from PATH	15 tests fail on a clean host, presenting as harness defects	✗	—	✓
A5	A benchmark scorer invoked bare `python`	score 0.0 manufactured by a missing interpreter	✓	✓	✓
A6	**Agent executed on the host; measurement in the pinned image**	the capstone — §5	✓	✓	✓

Class B — infrastructure failure conflated with measurement. The instrument's zero and the defect's zero are the same number.

#	Defect	Consequence	S	G	H
B1	Output marker used shell : (prints nothing)	a perfect 13/13 run scored as 13 errors	✓	✓	
B2	Results on stderr, markers on stdout	one framework's results fell outside the parsed slice	✓	✓	
B3	Probe diffed against a deliberately-deleted commit	every graded test a hole → "0 regressions"	✓	✓	
B4	Interrupted mirror clone accepted with zero commits	later instances die far from the cause	✓	✓	
B5	Container workdir unvalidated	every exec exit 127, OCI error on stdout	✓	✓	
B6	Isolation removed loopback, not just the internet	23.5% of the oracle dead; one instance 812 tests	✓	✓	
B7	Provider cache served force-removed containers	later cells of an instance replay against a corpse and **score clean**	✓	✓	
B8	Per-observation tree reset reverted image build-time edits	a repo family's oracle dead, disguised as flake	✓	✓	

Class C — final-state measurement of an event-shaped quantity. The number measured is not the quantity defined.

#	Defect	Consequence	S	G	H
C1	Bisection assumed monotone verdicts	recovered regressions invisible; recovering policy flattered	✓	✓	
C2	The declared event unit was implemented nowhere	pipeline counted per parametrised test; $\hat{\lambda}$ off by up to 2.7×	✓	✓	
C3	Rollbacks produced no observation	the recovering arm's recoveries invisible to the timeline scoring it	✓	✓	
C4	"Persists past the step boundary" defined nowhere	one legal reading excludes the events the design exists to count	✓	✓	

Class D — output consumed without asserting its producer.

#	Defect	Consequence	S	G	H
D1	Audit-trail field never populated	every checkpoint reported 0 tool errors, always	✓	✓	
D2	Malformed planner output crashed the run	33% of runs lost, scored as task failures	✗	✓	
D3	Dependency install counted as agent work	3,640 of 3,641 changed files; attribution vacuous	✓	✓	
D4	Join preferred the first observation over the last	failures pinned to the earliest tree under the fine grid	✓	✓	
D5	Executed config rebuilt from a name	manifest describes a run that never happened	✓	✓	
D6	Git handles never released	sweeps die of fd exhaustion after ~400 cells, loss concentrated on the back half	✓	✓	
D7	Console re-walked the full tree per run	answers first request, then presents as a dead server	✗	✓	
D8	Detection events never carried failing-test ids while attribution joined on them	attributed detection structurally UNKNOWN on every real run	✓	✓	
D9	Absent coverage rendered as measured silence	8 unmeasured episodes reported as attributed silence with unknown-rate 0.0	✓	✓	
D10	Turns that hit the output ceiling were executed anyway	truncated-but-parseable JSON wrote half a file; a 78-event storm read as agent incompetence, and the adapter remapped the stop reason that said why	✓	✓	

Totals, computed from the tables: **23 of 28 silent; 27 of 28 present under a green suite (the 28th being the suite itself); 6 of 28 findable only on a clean host.** D9 and D10 earn a sentence of humility: one appeared in the first valid run's own sidecar after an audit predicted it; the other on first contact with a second model family, whose longer outputs turned a latent ceiling into a mangled tree. The class does not exhaust.

4.2 The two design rules that caught most of them

Infrastructure failure is a missing observation, never a test failure — a typed `error`, never `fail`. This converted B1, B2, B3, B5 and B6 from fabricated regressions into visible holes.

Zero events must be distinguishable from a dead instrument. The baseline-liveness gate exists because an instrument whose every probe errored once scored a perfect negative control; on first contact with a clean host it failed loudly on A2 — the defect that would otherwise have been certified as a clean result.

4.3 The working practice that found the rest

Never consume a measurement's output without first asserting the measurement succeeded. D6 hid for weeks because a fixture consumed a sweep's output without checking the sweep's status. Nearly every row above is some component violating this rule; it is also how the official SWE-bench grader could be induced to mark unresolved patches resolved by forged stdout [5].

5 The capstone: every gate passed, and the number was still wrong

Three early pilots (80 runs, $99.07, development slice) measured an event rate far below the pre-declared gate. The gates had been fixed in advance; re-scoring confirmed detection had not regressed; we drafted the pre-declared conclusion — switch substrates. It was wrong. The harness ran the agent in a bare checkout on the host while every probe, verdict, and gate ran inside the pinned image; on the host `import matplotlib` in that checkout *succeeds* as an uncompiled namespace package, and everything downstream fails in ways indistinguishable from agent incompetence — 26 of 28 zero-step runs traced here. The rule behaved correctly on its data; the *inference* did not survive the data's provenance, because no gate had exercised the agent's execution path. **A validation gate certifies only the paths it exercises**; the fix is the seam the instrument already used for its probes: the agent must execute where it is measured.

5.1 What the corrected pipeline measures

All numbers exploratory, on a pre-declared development slice, excluded from any confirmatory frame. The declared event unit is one (test function, onset observation) pair, parametrised variants collapsed; we report **incidents** (distinct onset observations), **events** (declared unit), and **bearing runs** together, because the data are overdispersed and no single rate summarises them honestly.

Stack one — Claude planner/worker, 40 instances, rollback policy: resolve 32.5% (13/40, exact CI [18.6%, 49.1%]), and **zero events in 227 exhaustively-replayed observations** (bearing 0/40, CI [0%, 8.8%]). The same regime's earlier two-instance canary produced one bearing run (3 declared events), so the regime's pooled rate is near zero, not literally zero. The zero is demonstrated to belong to the runs: re-scoring that canary's archived timeline under this identical instrument reproduces its events exactly (§3).

Stack two — GPT-5.6 planner/worker, the same 40 instances, harness, policy, and budgets: resolve 37.1% (13/35 graded) at a quarter of the cost, and **140 declared events across 11 distinct incidents in 6 of 38 attempted runs** (bearing CI [6.0%, 31.3%]; the sweep's circuit breaker stopped the last two cells after six consecutive zero-progress failures in one repo family, and two cells died to a sync defect — all counted, none hidden). Against stack one's 0 of 40, the equal-N run-level contrast is significant: Fisher exact one-sided $p = 0.011$. Exposure does not explain it: observation densities differ by ~1.4×, event totals by 140-to-0.

Two disclosures. Stack two's first calibration was destroyed by defect D10 (a truncated write produced a 78-event storm; the circuit breaker stopped it at $1.12) and is excluded as infrastructure; the runs reported here show no capped turns in their event logs. And the pre-declared gate for this substrate still *fails* under stack two — $\hat{\lambda}$ clears its threshold, bearing 15.8% < 25% does not — so these numbers license not a confirmatory pass but the regime-conditional re-declaration of the gates, filed as an amendment before the contrast arms were unblinded.

What the events were. Every recovered event is invisible to final-state evaluation — the population prior work measures [3]. The harness's own view splits three ways: *detected and erased* (a failing check attributable to the regression, then rollback), co-occurring without an attributable link, and fully silent — 48 of the sweep's 162 raw episodes had no co-occurring harness failure of any kind. On the run that matters most, **an officially-resolved patch (all graded tests green) carried three regressions, two of which the harness never noticed.** One destructive edit broke 54 test functions at a single observation and left a perfect final state; a leaderboard cannot distinguish that run from one that never broke anything.

The undercount, measured across every bearing run in both sweeps:

	event-level record	visible in the final state
8 bearing runs, both sweeps	**184 declared events**	**1**

Final-state measurement — the instrument all published regression auditing uses [3] — captures 0.5% of what the timeline records here. The other 99.5% was created and repaired between the two states it examines.

5.2 The golden gate

The seam that closes A6 is validated by a *golden check*: the benchmark's own gold patch driven through the real routed tool path at $0 of model spend must grade resolved; a null run must not. Across the three hardest repo families it passed, catching three more defects before they could cost anything — one of which would have deflated every resolve rate ~3 points forever. On a rolling benchmark it surfaced **time-rotted oracles**: baseline tests failing in the raw image because the calendar passed dates baked into the instance. A benchmark's instances age even as its freshness defeats contamination.

5.3 The recovery-policy contrast, unblinded with its pre-committed analysis

Two more arms ran the same 40 instances under the event-producing stack: **repair-in-place** and **no recovery** (the failed step's unverified tree is kept). The paired analysis — exact two-sided sign tests

on final-state contamination (primary) and onset exposure (co-primary) — was committed before either arm finished.

arm	resolve	cells with final-state contamination	spend
rollback	13/35 = 37%	**1**	$8.14
repair-in-place	24/40 = 60%	5	$14.73
no recovery	22/40 = 55%	**9**	$4.40

Primary: no recovery leaves more contamination in the final state than rollback — worse on 9 paired instances, better on 1, ties 25; exact sign p = 0.022. Repair-in-place sits between (worse on 5, better on 1; p = 0.22). Exposure does not differ (co-primary p = 0.45): the damage happens under every policy; the policy decides what *persists*. One no-recovery tree ships with `$(cat django/db/models/expressions.py)` as line 1 of the module — a shell idiom pasted into a file write — and every one of its 137 graded tests missing; under rollback the same model's failed attempt on the same instance was reset and the run graded 137/137.

The tradeoff, reported because it cuts against our own founding thesis: **rollback resolves fewer tasks.** Against repair-in-place it wins 1 discordant pair and loses 9 (McNemar exact p = 0.022); against no recovery 3 and 9 (p = 0.15). The verifier gating the rollback is a planner-written check, and it rejects work the official grader accepts; rollback then destroys good patches on false rejections. Recovery policy trades resolution for cleanliness, and the exchange rate is set by verifier precision — a quantity no leaderboard reports.

One disclosure. The first unblinding reported the primary at p = 0.375. Seven cells — five no-recovery, two repair — had been dropped as "ungradable" because their final trees made the test suite uncollectable and the grader returned no verdict. Under official semantics that is every test failed, and those were the most catastrophic outcomes in the sweep, vanishing from the endpoint they were the strongest evidence for. The grader now disambiguates (a baseline run proves the environment alive) and the archived trees were re-graded with no model calls. Defect count at submission: twenty-eight in the census, and this one after it.

6 What this means for an agent OS

What to steal

1. **Type your silence** (§4.2, rule one) — in the schema, not in convention.
2. **Prove the instrument alive, not just non-lying** (§4.2, rule two), with positive controls that include *recovered* events.
3. **Never consume output without asserting the producer succeeded** (§4.3) — including your own ledger, fixtures, and event stream.
4. **First contact with a clean host is a validation step.** Six of our defects could not exist on the machine that wrote them.
5. **One environment.** The agent executes where it is measured, enforced by architecture. Our instrument had this seam for its probes and not for its agent; that asymmetry cost the most.

The taxonomy is the argument these rules generalise: the mechanisms are properties of subprocess-and-PATH, containers, parsers, and git — not of our code. Class A is OpenHands' issues #4235/#7044 [6]; class D is the SWE-bench grader consuming forgeable stdout [5]; UTBoost's oracle insufficiency [4] and §5.2's time-rotted oracles are class B one level up. And §5 gives the rules a stake: an OS layer that standardises recovery without standardising timeline observability will make agent runs look cleaner while hiding exactly what §5.3 measures.

7 Related work

SWE-bench [1] and Verified [2] are the substrate. TDAD [3] measures regressions from the final patch — the design §2 argues undercounts recovered events by construction; §5.1 measures the undercount directly: 184 events, 1 final-state-visible, across every bearing run in our data. UTBoost [4] audits the oracle itself. The grader's stdout-trust defect [5] and OpenHands' environment issues [6] are our classes D and A in the wild. SWE-bench-Live [7] addresses contamination with rolling instances

(median held-out oracle ~34× Verified's) and exhibits the aging-oracle failure §5.2 reports. Broader evaluation-infrastructure work [8] catalogues agent-eval irreproducibility; ours is mechanistic, and adds the first event-level regression measurements on this benchmark.

8 Limitations

Everything is exploratory and dev-slice: the contrast is one sweep per arm on the development slice, and the confirmatory held-out frame is registered but not run. Each cross-stack comparison is a single sweep per stack — sampling variability across re-runs of the same regime is unmeasured until the seeded replications in the confirmatory plan. The regime boundary includes the provider adapter, which near-par resolve rates bound but cannot eliminate. The oracle observes the gold test-patch's blast radius, so "regression" means regression adjacent to the edit; the observation hook fails open (its error counter is reported per cell). Binomial CIs treat the fixed dev slice as the population and ignore repository clustering (46% one repo). The catalogue is a census of one team's instrument; its external instances are evidence of reach, not a survey.

References

[1] Jimenez et al. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR 2024. arXiv:2310.06770.

[2] OpenAI. *Introducing SWE-bench Verified.* 2024.

[3] *TDAD.* arXiv:2603.17973.

[4] Yu et al. *UTBoost: Rigorous Evaluation of Coding Agents on SWE-Bench.* ACL 2025. arXiv:2506.09289.

[5] SWE-bench issue #601: *Test Result Hijacking via Stdout Forging in Evaluation Harness.*

[6] OpenHands issues #4235, #7044: environment errors in SWE-bench instance evaluation.

[7] Zhang et al. *SWE-bench Goes Live!* arXiv:2505.23419.

[8] *Holistic Agent Leaderboard.* arXiv:2510.11977.